

Course “Software Verification and Certification”

Course Code: CS 448/648

Term: *Spring 2026*

Lecture#5: 27 Jan 2026

“Verification Techniques: Model Checking Vs
Theorem Proving”

Dr. Gourinath Banda

Professor and Lead_PI - ACPS Lab

Department of Computer Science and Engineering

Indian Institute of Technology Indore

Venue @ L-01, LHC

- ***Formal Methods (HW/SW): Why They Exist & What Worked?***

- Learning Objective:
- Students shall understand--

Why formal methods became necessary

What “formal methods” actually mean

Which approaches succeeded in practice and why

Course Introduction [1/3]

- Term “Spring 2026”
- This course is an outcome of the project “Enabling Inclusive, Energy-Aware, and Formally Correct Computing: A Holistic Approach to Responsible System Development”

Thanks to “Mozilla Foundation, California, USA” for sponsoring this course design



1. Motivation: Complexity Is the Root Problem

CORE:

Systems will inevitably grow in **scale**, **functionality**, and **complexity**

--- making informal reasoning to fail.

Key points:

- Errors are no longer “just bugs”; they can have **catastrophic consequences**
- So manual/Traditional software engineering alone cannot cope!!!!
- *Formal methods aim to increase understanding, not magically guarantee correctness*

Important: “Formal methods do **not** guarantee correctness --- they expose ambiguity, inconsistency, and incompleteness.”

This positions verification as a “**knowledge amplifier**”, not a silver bullet.

Silver bullet: a simple, quick, and highly effective solution to a complex, difficult, or seemingly unsolvable problem

2. What Are Formal Methods?

Formal methods are:

- **Mathematically based languages**
- **Techniques**
- **Tools**
for **specifying and verifying systems**

What makes something “FORMAL”?

- Precisely defined **syntax** + Precisely defined **semantics** and
- Amenable to **mechanical analysis**.

If we have clarity then:

- Formal $\neq$ complicated and Formal $\neq$ unreadable;
- But Formal = **unambiguous**

3. “Specification” The First Success Story

Specification was where formal methods **first succeeded**.

Because:

- Writing things down precisely forces design clarity!
- Most errors are found/surface **before implementation**

Typically, there are two major styles of specification:

(a) State-based / Sequential specifications

Examples for this specification style languages:

- Z
- VDM
- Larch

Their characteristics include:

- Rich state spaces
- Pre/post conditions
- Functional correctness focus

(b) Concurrent / Reactive specifications

Examples:

- CSP (communicating sequential processes) , CCS (calculus of communicating systems)
- Statecharts
- **Temporal logic**
- **I/O automata : Timed Automata, Hybrid Automata**

Characteristics:

- Event-based behavior
- Ordering and concurrency
- Explicit temporal behavior

This is where **temporal properties** enter correctness.

4. Industrial Case Studies: Why Industry Finally Believed?

Recommended:

- **CICS (IBM + Oxford)** → cost reduction, fewer defects
- **CDIS (Air Traffic Management)** → no productivity loss, higher quality
- **TCAS II** → English spec failed, formal spec succeeded

Formal methods succeeded when **integrated**, not isolated.

Summary:

- Formal methods are **practical**
- They work best when embedded into engineering workflows

Lecture 4 : *Verification Techniques & the Road Ahead*

Learning Objective

Students shall understand:

- The difference between **model checking** and **theorem proving**
- **Is integration** necessary/beneficial/unavoidable?
- What future challenges still remain

1. Verification vs Specification (0–5 min)

Recall that:

- Specification: *What the system should do?*
- Verification: *Does the system satisfy the specification?*

This sets up the two major verification paradigms.

2. Model Checking: Automation and Counterexamples

Model checking in simple terms:

- 1. Build a **finite model**
- 2. Express properties (often temporal)
- 3. Exhaustively explore all behaviors

Key strengths of Model checking:

- 1. Fully automatic
- 2. Fast (less/ no manual intervention)
- 3. When ever a specification is not met, a Modelchecker provides a **Counter Example**.

Counter Example: a trace or sequence of actions/states in a system model that demonstrates a violation of a specified formal property

“This makes Model checking is primarily an **error-finding** technique.”

3. Theorem Proving: Expressiveness and Insight

Contrast with model checking:

Theorem proving:

- Handles **infinite state spaces**
- Uses **mathematical proof**
- Requires **human guidance**

Strengths:

- Deep correctness guarantees
- Suitable for arithmetic, data structures, algorithms

Weaknesses:

- Slower
- Expertise-heavy
- Less push-button

4. Integration: The pursuit

Explain their analogy:

- **Mixture vs Compound**
- Poor integration = methods coexist but don't interact
- Good integration = reasoning in one view affects another

Highlight:

- Model checking inside theorem provers
- Verification across lifecycle stages

5. Whats next in future?

Key themes:

- Abstraction and compositionality
- Reusable models
- Hybrid systems (discrete + continuous)
- Education as the bottleneck: Lack of trained engineers

A bright prospect is:

towards a future where engineers use formal methods **without noticing they are doing so.**

A Real Time System Example

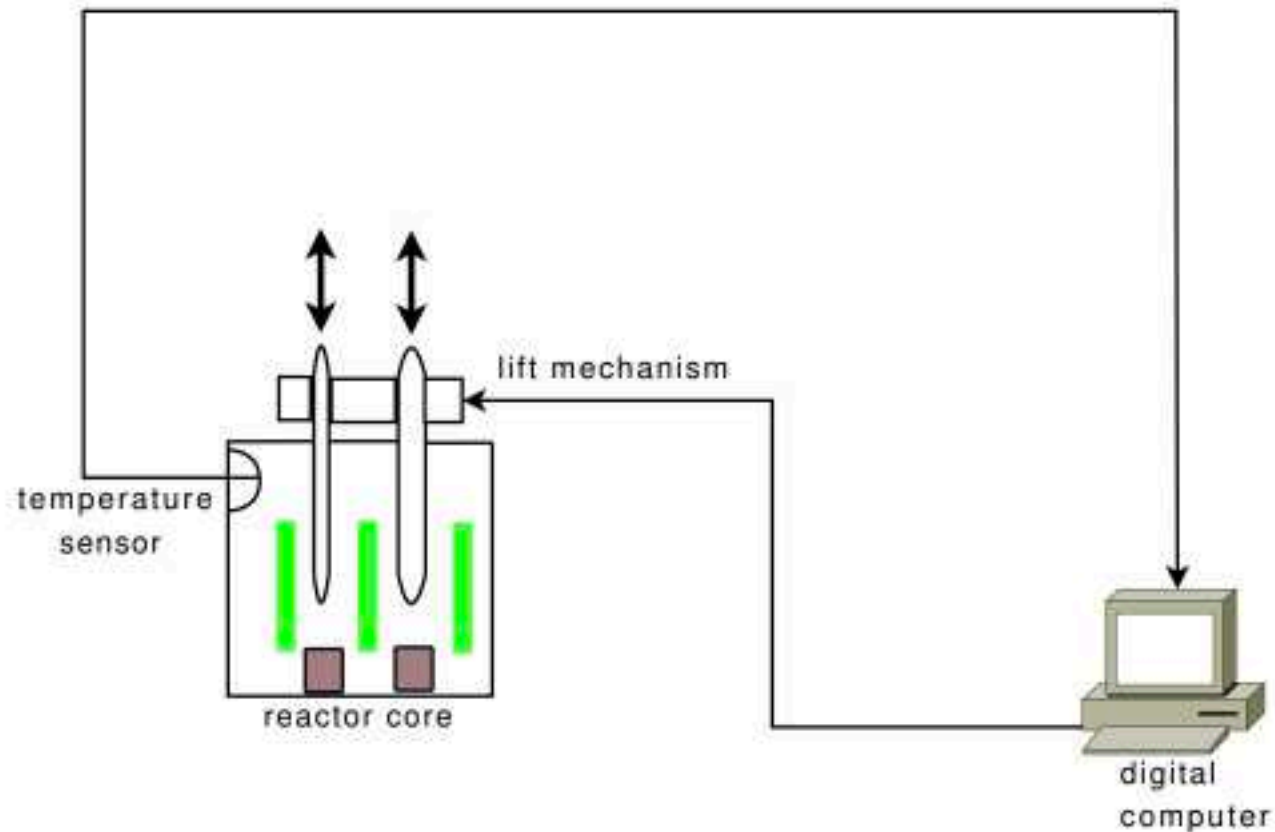


Figure: A (reactor) temperature control system

Linear Hybrid Automata (LHA)

- Hybrid Model: *Discrete Dynamics* + *Continuous Dynamics*
- It has:
 - locations and transitions (with guards)
 - discrete variables (labels)
 - cont. dynamics time invariant (such $dx/dt = k$ or $\dot{x} = k$)
 - invariants on locations (linear constraints)

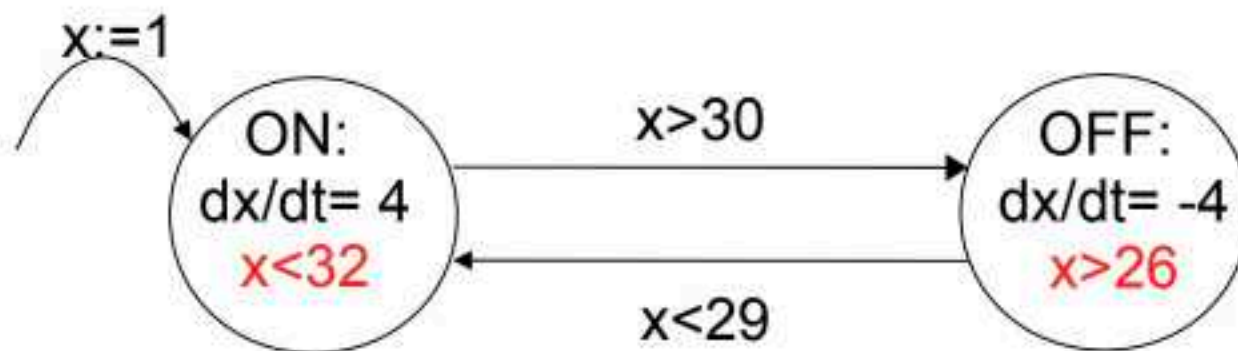


Figure: A simple LHA

LHA Syntax

An LHA is a 6-tuple $\langle \text{Loc}, \text{Trans}, \text{Var}, \text{Init}, \text{Inv}, \mathcal{D} \rangle$ where

- $\text{Loc} = \{l_0, \dots, l_n\}$: a finite set of locations;
- $\text{Var} = \{x_1, x_2, \dots, x_n\}$: a finite set of variables; Associated with Var are two sets:
 - $\dot{\text{Var}} = \{\dot{x}_1, \dots, \dot{x}_n\}$, where $\dot{x}_i = \frac{d}{dt}(x_i)$;
 - $\text{Var}' = \{x'_1, \dots, x'_n\}$.
 - Init , Inv and $\mathcal{D}$: functions assigning to $l \in \text{Loc}$ predicates $\text{Init}(l)$, $\text{Inv}(l)$ and $\mathcal{D}(l)$. $FV(\text{Init}(l))$ and $FV(\text{Inv}(l))$ range over Var , $FV(\mathcal{D}(l))$ range over $\dot{\text{Var}}$.
- $\text{Trans} = \{\tau_1, \dots, \tau_t\}$: a set of discrete transitions where $\tau_k = \langle k, l, \gamma_k, \alpha_k, l' \rangle$ is a transition
 - identified by integer k , $0 \leq k \leq t$;
 - corresponding to discrete transition l to l' ; and
 - guard γ_k and with actions α_k .

γ_k and α_k denote predicates $\gamma(k, l, l')$ and $\alpha(k, l, (l', X), (l', X'))$, respectively, where $X \subseteq \text{Var}$ and $X' \subseteq \text{Var}'$.

Temperature control system: LHA model

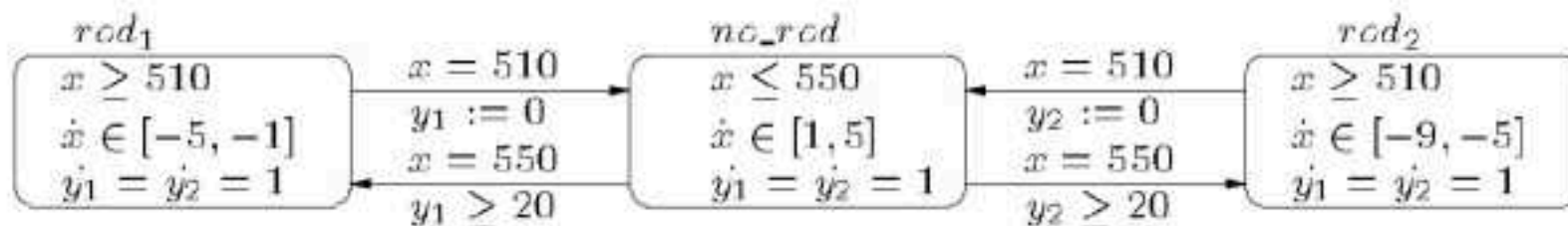
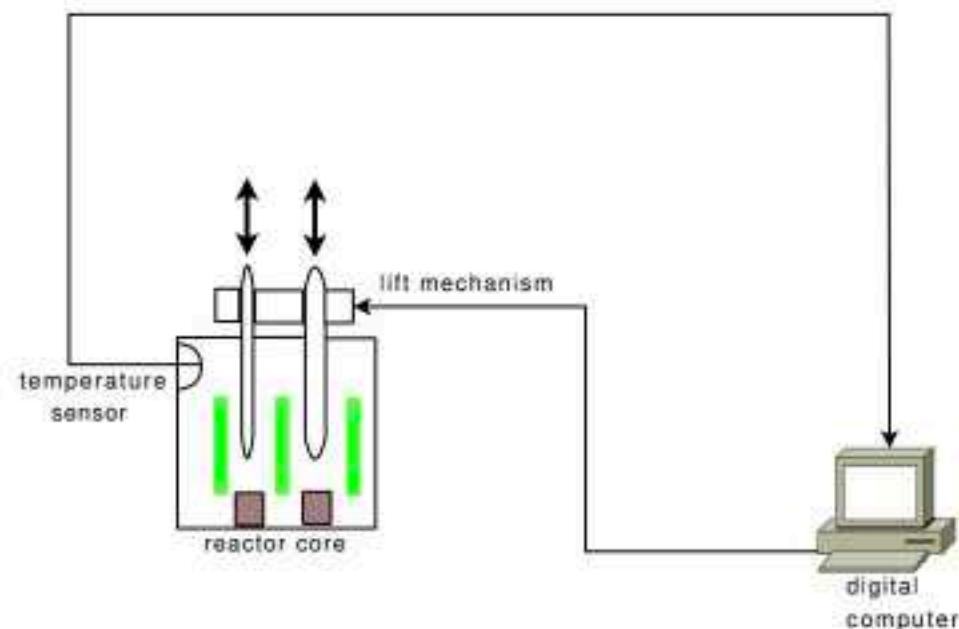


Figure: An LHA model

Water-level controller: LHA model

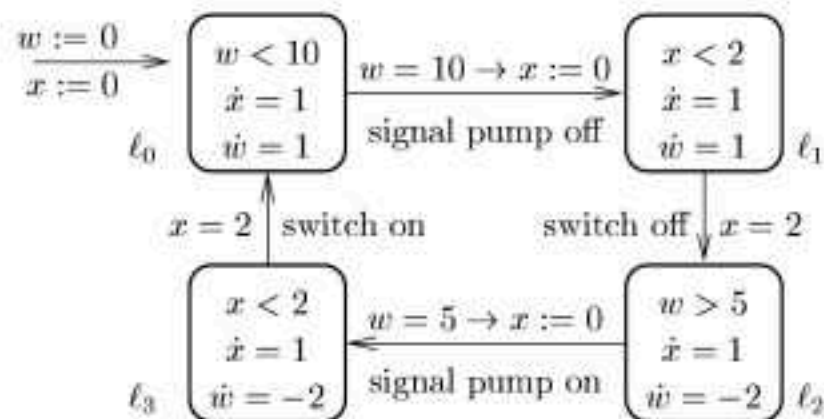
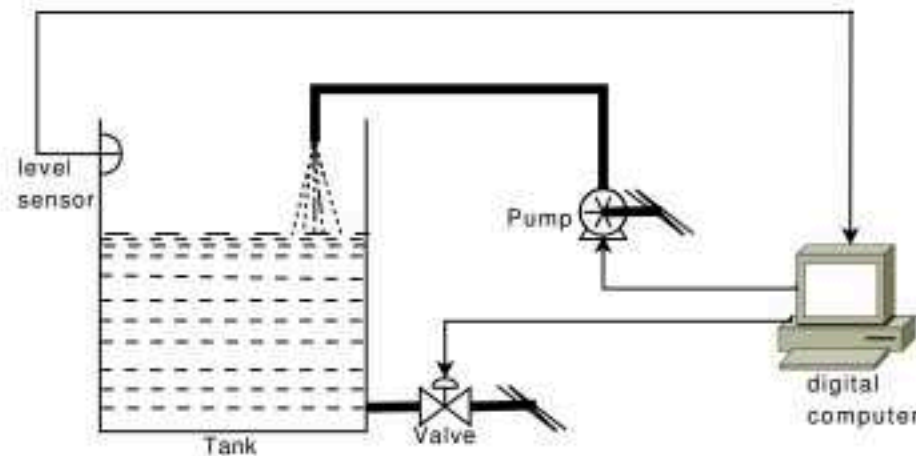
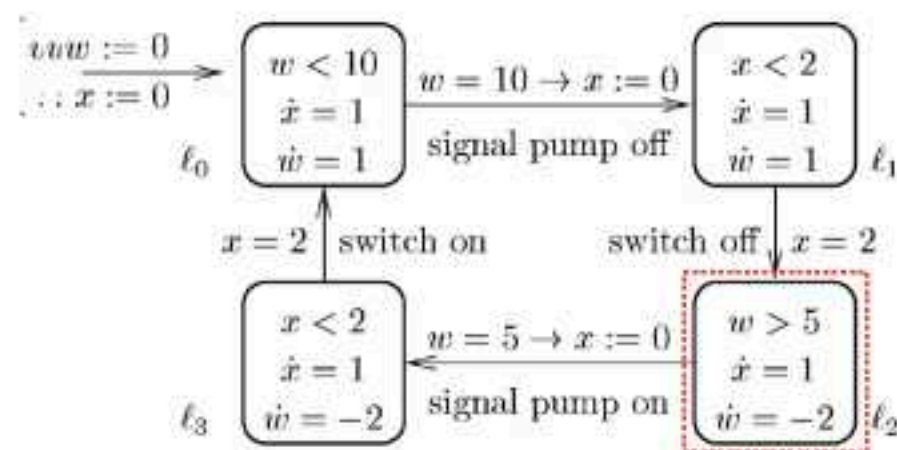


Figure: An LHA model

Run: Illustration

$$\text{Run: } \sigma = (l_0, X_0, t_0) \xrightarrow{(\gamma_0, \alpha_0)^{\delta_0}} (l_1, X_1, t_1) \xrightarrow{(\gamma_1, \alpha_1)^{\delta_1}} (l_2, X_2, t_2) \cdots$$

- Water-level controller:
 - $\sigma_w = (l_0, 0, 0) \dots \xrightarrow{(w=10, x=0)} (l_1, 10, 0) \dots \xrightarrow{(x=2, \{ \})} (l_2, w, 2) \dots$



- Verifying LHAs requires computing:
 - set of runs; (can verify any property)
 - set of reachable states (can do reachability analysis).

Computation Tree Logic (CTL)



- due to A.Pnueli 
- CTL is a property specification language for reactive systems.
- reactive properties are specified as the temporal ordering among the states.
- We can specify temporal ordering among events, states etc in CTL.
- Has Syntax and Semantics.

CTL Syntax

CTL Syntax in BNF

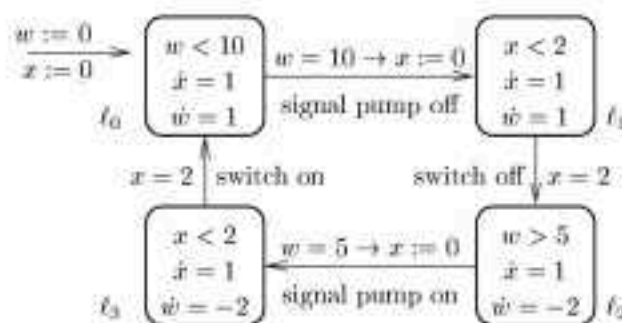
$$\begin{aligned} \phi ::= & \text{true} \mid p \mid \neg p \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \phi_1 \rightarrow \phi_2 \mid \phi_1 \leftrightarrow \phi_2 \mid \\ & AX\phi \mid EX\phi \mid AF\phi \mid EF\phi \mid AG\phi \mid EG\phi \mid \\ & AU[\phi_1, \phi_2] \mid EU[\phi_1, \phi_2] \mid AR[\phi_1, \phi_2] \mid ER[\phi_1, \phi_2] \end{aligned}$$

- liveness: eventually something **good happens**;
- safety: something **bad NEVER happens**;
- precedence: **temporal ordering** among the states.

property	universal	existential
safety	$AG \phi$	$EG \phi$
liveness	$AF \phi$	$EF \phi$
precedence	$AU[\phi_1, \phi_2]$	$EU[\phi_1, \phi_2]$

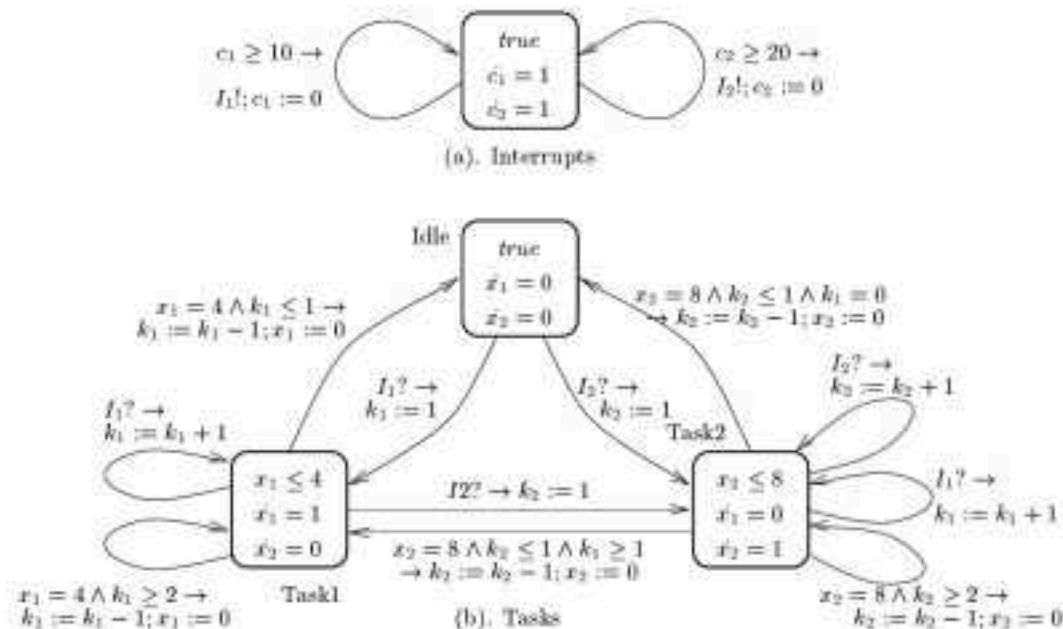
- ## Modelling

Example 1: Temporal properties of a water level controller [Halbwachs et al. 94]



- Safety property: $AG(0 \leq w \wedge w \leq 12)$. (w always stays between 0 and 12)
- Existential properties. $EF(w = 10)$. (w can reach the value 10).
- Eventual safety (nested CTL property).
 $AF(AG(1 \leq w \wedge w \leq 12))$. (Eventually, w remains between 1 and 12).

Example 2: Properties of a task scheduler [Halbwachs et al. 94]



- $AG(k2 > 0 \rightarrow AF(k2 = 0))$. (A waiting high priority task is eventually scheduled).
- $AG(k2 \leq 1)$. (No more than one high priority task can be waiting).

CTL Semantics

- The semantics of a (wff) CTL formula is its truth value.
- CTL formulas are evaluated w.r.t a Kripke structure.

Definition (Kripke Structure)

A Kripke structure K is a transition system

$\langle \text{States}, \text{InitStates}, \text{Trans}, \text{Prop}, \text{Label} \rangle$ where:

- States: (non-empty) set of states;
- $\text{InitStates} \subseteq \text{States}$: set of initial states;
- $\text{Trans} \subseteq \text{States} \times \text{States}$: *total relation* on States;
- Prop: set of atomic propositions *closed under negation* i.e.
if $p \in \text{Prop} \Leftrightarrow \neg p \in \text{Prop}$;
- $\text{Label} : \text{States} \rightarrow 2^{\text{Prop}}$ (**states** : $\text{Prop} \rightarrow 2^{\text{States}}$).

CTL Semantics (Trace-based)

Given K , ϕ evaluates to true if: $K, s \models \phi$. This satisfaction is inductively defined as:

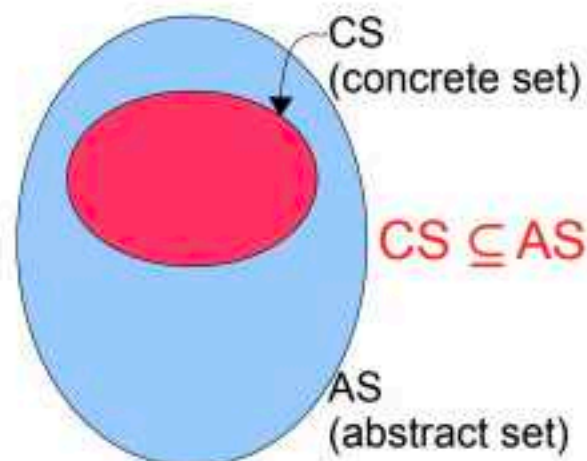
- $K, s \models \text{true}$ and $K, s \not\models \text{false}$
- $K, s \models p$ iff $s \in \text{states}(p)$
- $K, s \models \phi_1 \vee \phi_2$ iff $K, s \models \phi_1$ or $K, s \models \phi_2$
- $K, s \models AX\phi$ iff $\forall s' : (s, s') \in \text{Trans} : K, s' \models \phi$
- $K, s \models EX\phi$ iff $\exists s' : (s, s') \in \text{Trans} : K, s' \models \phi$
- $K, s \models AF\phi$ iff $\forall (s \ s_1 \ s_2 \ \cdots) : (\exists s_i : K, s_i \models \phi)$
- $K, s \models AG\phi$ iff $\forall (s \ s_1 \ s_2 \ \cdots) : (\forall s_i : K, s_i \models \phi)$
- $K, s \models AU[\phi_1, \phi_2]$ iff $\forall (s \ s_1 \ s_2 \ \cdots) : (\exists j \geq 0 : (K, s_j \models \phi_2 \wedge (\forall (0 \leq k < j) : K, s_k \models \phi_1)))$

$EF\phi$, $EG\phi$, $EU[\phi_1, \phi_2]$, $AR[\phi_1, \phi_2]$ and $ER[\phi_1, \phi_2]$.

Reachability Analysis (RA)

- certain properties could be verified by computing the set of *reachable states* (or *reaching states*).
- particularly: simple safety $AG\ p$ and liveness properties $EF\ p$.
- the set of reachable states **might NOT be computable always**.
- instead an **approximation** called *abstract state space* is computed.
- In abstraction-based reachability analysis:
 - possible to check $AG\ p$
 - possible to refute $EF\ p$

Concrete Vs. Abstract reachable states



- Safety $AG\ p$
 - holds iff:
 $\neg \exists s \in CS : s \models \neg p;$
- Liveness: $EF\ p$
 - fails iff:
 $\neg \exists s \in CS : s \models p;$

- Over-approximation RA:

(a) can only prove $AG\ p$

$\because AS \supseteq CS:$

$s_{\neg p} \notin AS \implies s_{\neg p} \notin CS$

(b) can only refute $EF\ p$

$\because AS \supseteq CS:$

$s_p \notin AS \implies s_p \notin CS$

- abstraction based methods are **not complete** (but **sound**)

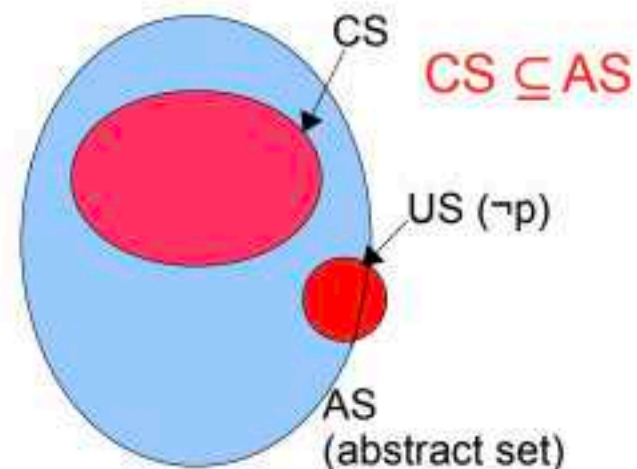
Abstraction-based reachability analysis

is incomplete, in this example:

- reports violation of safety ($AG\ p$)
- though it is not violated

Because:

$$(US \cap AS \setminus CS) \neq \emptyset$$



Such **possibly false alarms** can be exploited to refine:

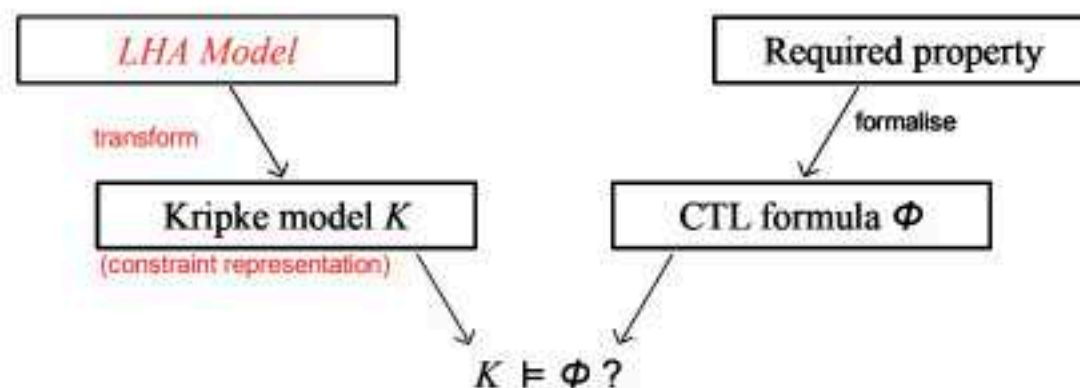
- the property; (refined property can be checked)
- the initialisation of an LHA; (refined system can be checked)

We propose such recipes (Section 5.4).

Model checking



- due to *Ed Clarke et. al*
- algorithmic technique to verify temporal properties of transition systems



- $K \models \phi$ i.e. $\forall s \in \text{InitStates} : s, K \models \phi$
 - 1 Compute $\llbracket \phi \rrbracket$, the set of states where ϕ holds;
 - 2 Check $\text{InitStates} \subseteq \llbracket \phi \rrbracket$.

Functions: states , $\text{pred}_{\exists}$, $\text{pred}_{\forall}$

- Given Kripke struc.

$K = \langle \text{States}, \text{Trans}, \text{InitStates}, \text{states}, \text{Prop} \rangle$:

- $\text{states} : \text{Prop} \rightarrow 2^{\text{States}}$
 - $\text{states}(p) = \{s \in S \mid p \in \text{Label}(s)\}$
- $\text{pred}_{\exists} : 2^{\text{States}} \rightarrow 2^{\text{States}}$
 - $\text{pred}_{\exists}(S') = \{s \mid \exists s' \in S' : (s, s') \in \text{Trans}\}$
- $\text{pred}_{\forall} : 2^{\text{States}} \rightarrow 2^{\text{States}}$
 - $\text{pred}_{\forall}(S') = \text{pred}_{\exists}(S') \setminus \text{pred}_{\exists}(\text{compl}(S'))$

